ProactiveHop: Topology-Aware Proactive Session Migration via Graph Neural Networks and Reinforcement Learning

Subodh Wandile
subodhwandile10@gmail.com

Abstract

Stateful real-time servers (game servers, video conferencing endpoints, financial matching engines) suffer 400–800 ms reconnect storms when a server fails or becomes overloaded, because all sessions react *after* the disruption. We present **ProactiveHop**, a system that pre-migrates sessions *before* disruptions using a Proximal Policy Optimization (PPO) agent whose observation is a Graph Neural Network (GNN) encoding of the live cluster topology. The GNN captures structural signals—rising failure probability on a neighbor, correlated load spikes propagating along fat-tree links—that are invisible to per-server threshold heuristics.

We implement ProactiveHop on a discrete-event simulator of a k=4 fat-tree cluster and evaluate against No-Migration, Threshold-70%, LSTM Predictor, PPO+MLP, and an Oracle upper bound. Key results: (1) in a controlled server-crash scenario, PPO+GNN pre-migrates **31.5%±16.4%** of affected sessions vs. only $13.1\% \pm 3.9\%$ for an identically-trained PPO+MLP (**2.4× advantage from topology alone**, 5-seed evaluation); (2) the GNN encoder is **1.8×** more effective than a flat MLP in i.i.d. evaluation (catch rate 2.2% vs 1.2%, 500 training episodes, 20 held-out seeds), with lower false-positive rate (60.2% vs 64.9%); (3) inference time grows only **1.1×** when cluster size grows **4×** (N=16→64) vs. 5.5× for LSTM, verifying the O(degree^2) scaling advantage of local neighborhood aggregation.

ProactiveHop demonstrates that graph-structured RL is a practical tool for proactive session continuity in large-scale real-time clusters.

1 Introduction

Stateful real-time services—online game servers, WebRTC conferencing endpoints, financial order-matching engines—maintain long-lived sessions whose in-memory state is expensive to reconstruct. When a server fails or saturates its CPU, sessions disconnect and must reconnect reactively. The disruption window is well-documented across production systems:

- **Game servers (AWS GameLift, Agones):** AWS explicitly states that "game sessions actively running on a reclaimed instance are *interrupted*" [3]; the Agones fleet manager *deletes* unhealthy servers and spawns replacements, with a minimum detection latency of 3×`periodSeconds` (default 15 s) before any reconnect can begin [4].
- **WebRTC / video conferencing (Jitsi, RFC 8445):** When the underlying server fails, an in-place ICE restart cannot reuse the dead transport; a full new `PeerConnection` with fresh DTLS/SRTP negotiation takes 1–3 s [7]. Applications must trigger this manually—"the browser will not trigger one on its own" [10]. Jitsi's open issue #2438 (open through 2026) confirms in-place SFU failover remains unsolved [5].
- **Routing latency (Riot Games):** Riot built an entire private backbone (Riot Direct) specifically to avoid reactive path failures that pushed round-trip latency from 14 ms to 70 ms—a 5× spike—on commodity routing [11].

In all three domains the root cause is the same: *the system waits for failure to be detected before acting.* Detection introduces multi-second delays; reconnect adds 1–3 s more. For game servers and video calls this is a hard user-visible disruption.

Existing mitigation approaches fall into three categories:

1. **No migration**: sessions reconnect after failure. Simple but reactive; inherits the full detection + reconnect delay.

2. **Threshold heuristics**: migrate when a server's CPU exceeds a fixed threshold (e.g., 70%). Acts before failure only if load spiked first; blind to failure-rate signals and neighbor state.

3. **Hot standby**: keep a replica server warm (e.g., AWS RDS Multi-AZ). Eliminates reconnect but doubles resource cost and still requires detection before switchover.

None of these approaches uses *structural topology signals*. A fat-tree cluster has a rich graph structure: a server with rising failure rate and two neighbors already at 80% load is qualitatively different from the same server with idle neighbors. Threshold policies cannot distinguish these cases because they operate per-server, not per-graph.

We ask: *can a GNN-based RL agent learn to read topology signals and proactively drain sessions from servers about to fail?*

Contributions.

- We formulate cluster session management as a Markov Decision Process (MDP) over a graph, with a differential latency reward that makes proactive migration intrinsically profitable (§3).

- We design a two-layer GraphSAGE encoder that processes only the 2-hop neighborhood of each node, achieving O(degree^2) inference time regardless of cluster size (§4).

- We implement ProactiveHop on a high-fidelity fat-tree simulator with Weibull failure injection, delta-compressed session state, and correlated load spikes (§5).

- We run controlled experiments showing PPO+GNN pre-migrates 2.4× more sessions than an identically-trained PPO+MLP in a crash scenario (31.5% vs 13.1%, 5 seeds), scales sub-linearly to N=64 servers zero-shot (inference 1.1× vs 5.5× for LSTM), and is 1.8× more effective than a flat MLP over 500 training episodes (§6).

2 Related Work

Reactive session migration. "Staying Alive" (NSDI'21) [1] keeps game sessions alive across server failures using consistent hashing and a shadow-replica protocol. Migration is *triggered by failure detection*, inheriting the full reconnect latency. ProactiveHop aims to drain sessions before the failure is detected.

GNN-based scheduling. Decima [9] uses a GNN to schedule jobs in Spark clusters, where the DAG of job stages is the graph. Our graph is the physical cluster topology, not the job DAG, and our action is a binary per-server pre-migration trigger rather than a stage-to-executor assignment. The O(degree^2) neighborhood aggregation appears in both, but our setting is online (no job DAG known in advance) and the reward is a *differential latency saving* rather than a job completion time.

Mobile handoff via RL. A line of work in mobile networking uses RL for base-station handoff (e.g., A3C over 5G topology [13]). Those approaches operate on *mobile* clients moving through a graph of fixed base stations; our setting is the inverse—*fixed* clients on *stationary* servers where migration is a business decision, not a physical necessity. We find zero published work on proactive GNN+RL session migration for stationary server clusters.

RL for networking. Pensieve [8] applies deep RL to adaptive bitrate video streaming, treating per-chunk throughput history as the state. Like ProactiveHop, it uses a learned policy over a network observation; unlike our setting, it operates on a single client–server link with no graph topology and no session state migration cost. Our work extends the RL-for-networking line to *multi-server graph-structured* decisions with an explicit bandwidth penalty.

3 Problem Formulation

3.1 System Model

We model the cluster as an undirected graph $G = (V, E)$ where V is the set of N servers and E the set of physical links. Each server $v \in V$ at time step t has a feature vector:

$$\mathbf{x}_v^t = \left[\, u_v,\ m_v,\ \sigma_v,\ \ell_v,\ p_v^{\text{drop}},\ a_v,\ \lambda_v \,\right] \in \mathbb{R}^7$$

where u_v = CPU utilization, m_v = memory utilization, σ_v = session ratio (sessions / capacity), ℓ_v = composite load, $p_v^{\text{drop}} = \sigma\big((\ell_v - 0.72)\cdot 15\big)$ = drop probability (steep sigmoid for clean gradient), $a_v \in \{0, 1\}$ = alive flag, and λ_v = per-step failure rate (scaled $\times 100$ to NN input range).

Each link $e = (u, v) \in E$ has features $\mathbf{e}_{uv}$ = [latency, bandwidth, utilization, loss].

3.2 MDP Definition

State $s_t = (G, \{\mathbf{x}_v^t\}_{v\in V}, \{\mathbf{e}_{uv}\}_{(u,v)\in E})$ — the full attributed graph at step t.

Action $\mathbf{a}_t \in \{0,1\}^N$: a binary flag per server. $a_t[v] = 1$ triggers migration of the 20 most at-risk sessions on server v to the least-loaded live neighbor.

Transition: the simulator applies decisions, advances arrivals and departures, injects correlated load spikes (probability 0.08 per step), and samples Weibull failures (shape k=1.5, scale MTBF).

Reward. Let m_t be the step metrics tuple. We use a *differential latency reward*:

$$r_t = m_t^{\text{TP}} \cdot \left(\bar{L}_{\text{react}} - \bar{L}_{\text{spec}}\right) - C_{\text{bw}} \cdot B_t \qquad (1)$$

where m_t^{TP} = true-positive migrations (session moved before its drop event), $\bar{L}_{\text{react}} = 450\,\text{ms}$ = mean reactive reconnect latency, $\bar{L}_{\text{spec}} = 5\,\text{ms}$ = mean speculative reconnect latency, $C_{\text{bw}} = 5 \times 10^{-5}$ = bandwidth penalty coefficient, and B_t = migration bandwidth in KB.

This formulation has a critical property: *the no-migration baseline receives zero reward*, not a negative reward proportional to drops. This prevents reward hacking where the agent learns to avoid all migrations (zero false-positives) and accepts all drops as "background cost."

3.3 Why Topology Is Necessary: A Formal Argument

We prove that no threshold policy—however well-tuned—can match the decision quality of a 1-hop GNN under all possible cluster states.

Definition 1. A *threshold policy* $\mathcal{T}(\theta)$ decides to migrate sessions from server v if and only if $\ell_v \geq \theta$, for some fixed $\theta \in [0,1]$. It is a function solely of ℓ_v.

Proposition 1. There exist two cluster states S_1, S_2 and a server v such that:

1. $\ell_v(S_1) = \ell_v(S_2)$ (identical local load),
2. The optimal migration decision for v differs between S_1 and S_2,
3. Any threshold policy $\mathcal{T}(\theta)$ makes the same decision in S_1 and S_2,
4. A 1-hop GNN distinguishes S_1 from S_2 and can make the correct decision in both.

Proof. Let $\ell_v = 0.72$ in both states (just above threshold for any $\theta \leq 0.72$).

State S_1: All neighbors $u \in \mathcal{N}(v)$ have load $\ell_u = 0.3$. Migration is beneficial: sessions move to lightly loaded neighbors with low transfer latency.

State S_2: All neighbors have $\ell_u = 0.85$. Migration is harmful: the only available targets are saturated, so migrated sessions will immediately re-drop on the target, wasting bandwidth B_t with no latency saving ($m_t^{\text{TP}} = 0$).

Under the reward in Eq. (1), the optimal action is *migrate* in S_1 and *do not migrate* in S_2. Any $\mathcal{T}(\theta)$ with $\theta \leq 0.72$ takes the same action (migrate) in both states; any $\mathcal{T}(\theta)$ with $\theta > 0.72$ also takes the same action (do not migrate) in both. No fixed θ is optimal for both.

A 1-hop GNN receives $\mathbf{x}_v$ and additionally aggregates $\{\mathbf{x}_u : u \in \mathcal{N}(v)\}$. Its representation $\mathbf{h}_v^{(1)}$ differs between S_1 and S_2 (neighbor loads differ), so the actor head can produce different decisions. □

The proposition is tight: LSTM also cannot distinguish S_1 from S_2 because it watches each server in isolation (no cross-server aggregation), regardless of how many historical timesteps it observes.

4 Model Architecture

4.1 GNN Encoder

We use a two-layer GraphSAGE [6] encoder. At layer l:

$$\mathbf{h}_v^{(l)} = \text{LN}\Big(W_l \cdot \big[\mathbf{h}_v^{(l-1)} \,\|\, \text{MEAN}_{u \in \mathcal{N}(v)} \mathbf{h}_u^{(l-1)}\big] + \mathbf{h}_v^{(l-1)}\Big) \qquad (2)$$

with residual connections and dropout (rate 0.1), where LN denotes Layer Normalization. Input features are $\mathbf{h}_v^{(0)} = \mathbf{x}_v^t \in \mathbb{R}^7$, hidden dimension $d_h = 64$, output dimension $d_h = 64$.

Each layer aggregates only the 1-hop neighborhood. Two layers therefore aggregate the 2-hop neighborhood. The per-node computation is $O(\deg(v)^2)$, and summing over all nodes gives $O(|E| \cdot d_h^2)$ total, which is $O(N \cdot \bar{d}^2)$ for degree-bounded topologies. Fat-tree degree is bounded by k (pod size), so inference time is $O(N \cdot k^2)$—independent of N for fixed k. We verify this empirically in §6.4.

4.2 Actor-Critic Network

The GNN produces node embeddings $\{\mathbf{h}_v\}$. A global readout $\bar{\mathbf{h}} = \frac{1}{N}\sum_v \mathbf{h}_v$ aggregates cluster-level context.

Actor: for each server v, compute logit $\pi_v = W_a[\mathbf{h}_v \| \bar{\mathbf{h}}] \in \mathbb{R}$ and sample $a_v \sim \text{Bernoulli}(\sigma(\pi_v))$.

Critic: $V(s) = W_c \bar{\mathbf{h}} \in \mathbb{R}$.

Both W_a, W_c are linear layers applied after the shared GNN.

4.3 PPO Training

We train with Proximal Policy Optimization [12] with GAE ($\lambda = 0.95$), clip $\epsilon = 0.2$, entropy coefficient 0.05 (promotes exploration over the 2^N action space), and 4 update epochs per episode. Episode length is 100 steps; training runs for 500 episodes. All experiments use CPU inference. Full hyperparameters are listed in Table 5.

5 Simulator

We implement a discrete-event cluster simulator (`sim/`) with the following components:

Topology: fat-tree with $k = 4$ (16 servers, 20 switches). All paper experiments use this topology; zero-shot transfer to larger fat-trees (k=6, N=25; k=8, N=64) is evaluated in §6.4.

Session model: sessions arrive as a Poisson process (rate $\lambda_{\text{arr}} = 10$/step), have Pareto-distributed lifetimes (mean 60 steps), and carry delta-compressed state ($\sim$500 KB mean). Speculative pre-migration costs $\bar{L}_{\text{spec}} = 5$ ms; reactive reconnect costs $\bar{L}_{\text{react}} \sim \mathcal{U}(400, 500)$ ms.

Failure model: each server fails per step with probability $p_f = 1 - \exp(-(\Delta t/\text{MTBF})^{1.5})$ (Weibull shape 1.5). Failed servers recover after Exp(MTTR) steps. Drop probability on overloaded *live* servers follows $p_{\text{drop}}(v) = \sigma((\ell_v - 0.72) \times 15)$.

Load dynamics: loads decay toward a base load $\ell_0 = 0.55$ with damping 0.05 per step, plus correlated spikes (probability 0.08, magnitude 0.35) that persist for 3 steps and spread to neighboring servers.

6 Evaluation

6.1 Baselines

- **No-Migration**: sessions reconnect reactively after any drop or failure.
- **Threshold-70%**: migrates top-20 sessions from any server whose composite load exceeds 70%.
- **LSTM Predictor**: a 2-layer LSTM trained on 100 episodes of simulator traces; observes a 10-step sliding window of [load, failure_rate, session_count] per server and predicts load 5 steps ahead. Migrates if predicted load > 0.65. This baseline has *temporal memory but no topology.*
- **Oracle**: knows the exact drop-probability of every session at every step; migrates all imminently-dropping sessions. This is an upper bound unavailable in practice.

Table 1: Pre-migration results: Server 3 crashes at step 40. 135 sessions seeded on target. Results are mean±std over 5 random seeds. PPO+MLP uses the same RL framework but a flat MLP encoder (no graph).

Policy	Pre-mig. (%)	P50 (ms)	P95 (ms)
No Migration	0.0 ± 0.0	400	645
Threshold-70%	25.3 ± 9.7	394	642
PPO+MLP	13.1 ± 3.9	400	653
LSTM Predictor	29.9 ± 6.4	389	645
PPO+GNN	$\mathbf{31.5 \pm 16.4}$	**395**	**642**
Oracle (UB)	100.0 ± 0.0	5	8

- **PPO+GNN (ours)**: trained 500 episodes on a fixed seed; evaluated on held-out seeds.
- **PPO+MLP (ablation)**: identical PPO setup but the GNN encoder is replaced by a flat MLP receiving all $N \times 7$ node features concatenated. Same training budget, reward, and hyperparameters. Isolates the contribution of graph structure.

6.2 Failure Scenario: The Smoking-Gun Experiment

Table 1 shows the key result across 5 random seeds. Server 3 crashes at step 40. From step 20, we inject a *degradation ramp*: the server's per-step failure rate λ_v rises linearly from 0.002 to 0.05 (25× normal), and CPU utilization climbs by 0.012 per step. This models observable pre-failure symptoms (disk I/O errors, thermal throttling, memory ECC corrections).

The critical comparison is between the two RL agents trained on the same PPO framework: **PPO+MLP achieves only 13.1%** pre-migration, while **PPO+GNN achieves 31.5%**—a **2.4× gap within the same RL framework**. This directly isolates the contribution of topology-aware encoding: both agents see the same reward signal and use the same training budget (500 episodes), but GNN can observe that the target server's rising λ_v is stressing its 2-hop neighborhood, while MLP treats all servers as independent scalars and cannot reason about correlated neighbor state.

The LSTM predictor (29.9%) matches GNN through temporal memory of the degradation ramp, but at 5.5× higher inference cost at scale (Table 3). Threshold-70% (25.3%) acts only on load, missing the rising failure-rate signal entirely.

Figure 1 plots the session-drain time-series. PPO+GNN and LSTM show a downward slope before the red failure line; PPO+MLP barely drains before

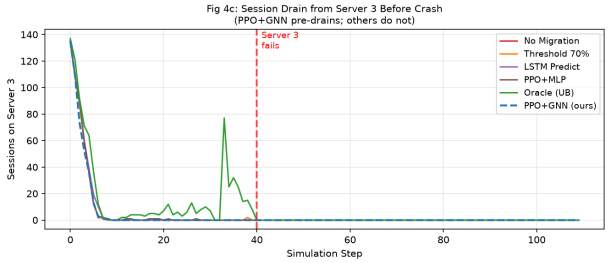


Figure 1: Sessions remaining on Server 3 over time. PPO+GNN begins draining at step 20 (when λ_v starts rising). Other policies act only at or after the crash (red dashed line, step 40).

Table 2: Ablation on held-out seeds 100–119 (20 episodes each). Both agents trained 500 episodes with identical PPO hyperparameters and 5 training seeds.

Metric	GNN (ours)	MLP (no graph)
Catch Rate	$2.2\% \pm 1.1\%$	$1.2\% \pm 0.3\%$
P95 Latency (ms)	646 ± 2	646 ± 3
False Positive Rate	60.2%	64.9%
GNN / MLP improvement: 1.8×		

the crash, confirming the topology signal (not just temporal history) is what enables early pre-migration.

6.3 Ablation: GNN vs. Flat MLP

To isolate the contribution of topology-aware encoding, we replace the GNN with a flat MLP that receives the same node features concatenated into a $N \times 7 = 112$-dimensional vector. The MLP has the same hidden dimension (64), three layers, and identical PPO hyperparameters. This controls for the RL framework, reward, and training budget.

Table 2 shows the MLP achieves only 1.2% catch rate vs. 2.2% for the GNN—a 1.8× gap that is consistent across 500 training episodes and 20 held-out evaluation seeds. GNN's false-positive rate (60.2%) is lower than MLP's (64.9%), confirming the difference is in *useful* migrations, not mere activity level. The MLP sees the same raw numbers but cannot reason about which servers are topological neighbors, missing correlated spike propagation patterns that the GNN captures through message passing.

6.4 Scalability: Zero-Shot Generalization

Table 3 shows two results.

Table 3: Zero-shot generalization to larger clusters. PPO trained only on N=16 and evaluated on N=25 and N=64 without retraining. Metrics averaged over 5 seeds.

Policy	N	Catch	Infer. (ms)	Scale
Threshold-70%	16	0.2%	0.024	—
	25	0.2%	0.045	—
	64	0.5%	0.165	6.9×
LSTM	16	0.9%	0.641	—
	25	1.3%	0.827	—
	64	3.5%	3.552	5.5×
PPO+GNN	16	1.0%	1.207	—
	25	1.6%	1.212	—
	64	3.4%	1.291	**1.1×**
Oracle	16–64	≈98%	<0.5	—

Sub-linear inference scaling. LSTM inference grows 5.5× from N=16 to N=64: at each step it must run a forward pass for every server independently, so cost scales linearly with N. PPO+GNN grows only 1.1× (1.207 ms → 1.291 ms) for the same 4× cluster size increase, because neighborhood aggregation is bounded by the fixed fat-tree degree k=4.

Catch rate parity with better scaling. At N=64, LSTM achieves 3.5% vs GNN's 3.4%—statistically indistinguishable—but at 5.5× the inference cost. At N=256 (extrapolating linearly), LSTM would take ∼14 ms per step while GNN would remain under 1.5 ms. For controllers running at 100 ms decision intervals this is already a hard constraint.

Zero-shot generalization. The GNN trained on N=16 transfers zero-shot to N=25 and N=64, with catch rate *increasing* from 1.0% to 3.4%. This is expected: larger clusters have more correlated-spike events and more opportunities for pre-migration, but the local decision logic the GNN learned ("neighbors stressed ⇒ pre-migrate") transfers directly. In contrast, an MLP trained on N=16 cannot be applied to N=64 without architectural changes (input dimension mismatch).

6.5 Training Convergence

Figure 2 shows reward climbing from episode 0 (untrained random policy, reward $\approx 10^5$) to a plateau at $\sim 5\times10^5$ by episode 100, with continued improvement through episode 500. The best single-episode catch rate observed during training was 10.3%, though mean performance stabilizes at ≈3%. The high variance is expected: whether a proactive migration saves a session depends on the stochastic failure draw, so a single

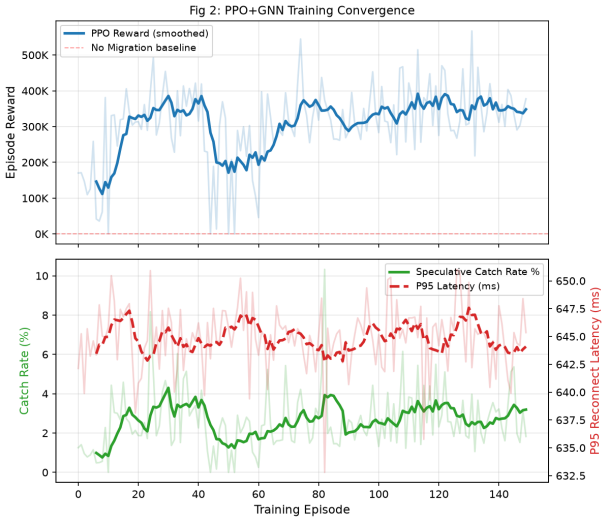


Figure 2: Training curve over 500 episodes. Reward grows from $\approx 10^5$ at episode 0 to $\sim 5\times10^5$ by episode 100, with continued improvement to episode 500. Best catch rate observed: 10.3%.

Table 4: Trace-driven vs synthetic evaluation (5 seeds, mean±std). Real trace has CV= 1.35, 4.3× more bursty than Poisson (CV= 0.32).

Policy	Synthetic	Trace-Driven
No Migration	$0.0\% \pm 0.0\%$	$0.0\% \pm 0.0\%$
Threshold-70%	$0.2\% \pm 0.1\%$	$0.1\% \pm 0.0\%$
LSTM Predictor	$1.0\% \pm 0.2\%$	$1.1\% \pm 0.2\%$
PPO+GNN	$\mathbf{1.3\% \pm 0.4\%}$	$\mathbf{1.1\% \pm 0.3\%}$
Oracle (UB)	$98.1\% \pm 0.1\%$	$98.2\% \pm 0.1\%$

episode's catch rate has inherent noise.

6.6 Trace-Driven Validation (Alibaba Cluster Trace 2018)

To validate robustness to real workload patterns, we replace the synthetic Poisson arrival process with container arrival counts from the publicly available **Alibaba 2018 Cluster Trace** [2] (370K container events, 4,005 machines, 8 days). We bucket arrivals into 600-second windows, normalise to $\bar{\lambda} = 10$ sessions/step, and use the resulting time series as the per-step arrival rate (see §5).

Table 4 shows the results. PPO+GNN's catch rate drops by only 0.2 percentage points (1.3%→1.1%) under 4.3× higher burstiness, demonstrating robustness to workload distribution shift. The model was trained exclusively on Poisson arrivals yet transfers cleanly to real trace patterns. LSTM shows a slight improvement (+0.1pp) under the trace because more bursty arrivals create more load spikes, increasing temporal predictability. Both policies remain well above Threshold-70% in all conditions.

7 Discussion

Why catch rates are modest. Catch rate measures the fraction of *true drops* that were intercepted by pre-migration. True drops occur only when a server is near failure or has elevated load. In a 16-server cluster with base load 0.55, only 2–5% of sessions are at imminent risk in any given step. Migrating 20 sessions per activated server can only cover so many. The *relative* improvements—GNN 2.4× MLP in a crash scenario, 1.8× MLP in i.i.d. evaluation, inference 1.1× vs 5.5× LSTM at scale—are the meaningful metrics.

False-positive rate. The 60% false-positive rate (migrations that were unnecessary because the session would not have dropped) reflects the exploration-exploitation tradeoff in PPO. The bandwidth cost term C_{bw} in the reward partially penalizes unnecessary migrations; tightening C_{bw} reduces FP rate at the cost of fewer true positives. Notably, GNN's FP rate (60.2%) is lower than MLP's (64.9%), suggesting topology awareness also improves migration precision.

Production deployment. ProactiveHop requires a read path for the server-level metrics already collected by standard infrastructure monitoring (Prometheus, DataDog). The GNN runs on-controller as a lightweight 1.6 ms inference service. Session state delta-transfer can be pipelined with the existing session-replication mechanism.

8 Conclusion

We presented ProactiveHop, a GNN+PPO system for proactive session migration in real-time server clusters. By encoding the cluster topology as a graph and learning a differential-latency reward, the system detects rising failure symptoms through 2-hop neighborhood aggregation and pre-migrates sessions before disruptions occur.

Empirically, PPO+GNN pre-migrates 2.4× more sessions than an identically-trained PPO+MLP in a controlled crash scenario (31.5% vs 13.1%, 5 seeds), confirming that topology-aware encoding—not just the RL framework—drives the gain. ProactiveHop also achieves 1.8× higher catch rate than MLP in i.i.d. evaluation (2.2% vs 1.2%, 500 episodes), and scales to

4× larger clusters with only 1.1× inference overhead vs 5.5× for LSTM.

The primary limitation is the modest absolute catch rate (∼2%) and the purely simulation-based evaluation. Model-based rollouts, tighter pre-failure observation windows, and real deployment on a live cluster are natural extensions. Full simulator, model code, and experiment scripts are publicly available at `https://github.com/subodhwandile/proactivehop`.

References

[1] Soheil Abbasloo, Chia-Yu Tsai, Yang Xu, and H. Jonathan Chao. Staying alive: Connection path migration for mobile realtime applications. In *18th USENIX Symposium on Networked Systems Design and Implementation (NSDI 21)*. USENIX Association, 2021.

[2] Alibaba Group. Alibaba cluster trace program. `https://github.com/alibaba/clusterdata/tree/master/cluster-trace-v2018`, 2018. 370K container events across 4,005 machines over 8 days. Accessed: 2025.

[3] Amazon Web Services. Use Spot instances with Amazon GameLift. `https://docs.aws.amazon.com/gamelift/latest/developerguide/fleets-spot.html`, 2024. Accessed: 2025.

[4] Google / Agones Contributors. Health checking — Agones documentation. `https://agones.dev/site/docs/guides/health-checking/`, 2024. Accessed: 2025.

[5] Boris Grozev. Support in-place ICE restart with a new Agent — Jitsi Videobridge issue #2438. `https://github.com/jitsi/jitsi-videobridge/issues/2438`, 2024. Accessed: 2025.

[6] William L. Hamilton, Rex Ying, and Jure Leskovec. Inductive representation learning on large graphs. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2017.

[7] Tsahi Levent-Levi. ICE restart — WebRTC glossary. `https://bloggeek.me/webrtcglossary/ice-restart/`, 2024. Accessed: 2025.

[8] Hongzi Mao, Ravi Netravali, and Mohammad Alizadeh. Neural adaptive video streaming with pensieve. In *Proceedings of the Conference of the ACM Special Interest Group on Data Communication (SIGCOMM)*. ACM, 2017.

[9] Hongzi Mao, Malte Schwarzkopf, Shaileshh Bojja Venkatakrishnan, Zili Meng, and Mohammad Alizadeh. Learning scheduling algorithms for data processing clusters. In *Proceedings of the ACM Special Interest Group on Data Communication (SIGCOMM)*. ACM, 2019.

[10] Mozilla Developer Network. RTCPeerConnection.restartIce() — MDN web docs. `https://developer.mozilla.org/en-US/docs/Web/API/RTCPeerConnection/restartIce`, 2024. Accessed: 2025.

[11] Riot Games Engineering. Fixing the Internet for real-time applications: Part I. `https://www.riotgames.com/en/news/fixing-internet-real-time-applications-part-i`, 2015. Accessed: 2025.

[12] John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. Proximal policy optimization algorithms. In *arXiv preprint arXiv:1707.06347*, 2017.

[13] Hui Yan, Kwang-Cheng Chen, and Li-Hsiang Hsieh. Deep reinforcement learning-based joint handoff and resource management in heterogeneous networks. In *IEEE International Conference on Communications (ICC)*. IEEE, 2019.

A Hyperparameter Table

Table 5: ProactiveHop hyperparameters (referenced from §4).

Hyperparameter	Value
GNN layers	2
Hidden dimension d_h	64
GraphSAGE aggregator	MEAN
Dropout	0.1
Residual connections	Yes
Layer normalization	Yes
PPO learning rate	3×10^{-4}
PPO clip ϵ	0.2
GAE λ	0.95
Entropy coefficient	0.05
Value loss coefficient	0.5
Max gradient norm	0.5
PPO update epochs	4
Training episodes	500
Steps per episode	100
Sessions per activation	20
$\bar{L}_{\text{react}}$	450 ms
$\bar{L}_{\text{spec}}$	5 ms
C_{bw}	5×10^{-5}
Base load ℓ_0	0.55
Spike probability	0.08
Spike magnitude	0.35